SHIA-RAG: A Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

Major Project-I (IT898) Report
Submitted in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY
in
INFORMATION TECHNOLOGY

by

Kshitij Khede
(252IT010)

under the guidance of

Prof. Ananthanarayana V. S.



DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALORE - 575025
September, 2026

DECLARATION

I hereby *declare* that the Report of the P.G. Major Project-I Work entitled "SHIA-RAG: A Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation", which is being submitted to the National Institute of Technology Karnataka, Surathkal, in partial fulfillment of the requirements for the award of the Degree of Master of Technology in Information Technology in the Department of Information Technology, is a *bonafide report of the work carried out by me.* The material contained in this report has not been submitted to any University or Institution for the award of any degree.

Kshitij Khede (252IT010)
Dept. of Information Technology
NITK Surathkal, Mangalore

Place: NITK, Surathkal
Date:

CERTIFICATE

This is to *certify* that the P.G. Major Project-I Work entitled "SHIA-RAG: A Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation", submitted by Kshitij Khede (252IT010) as the record of the work carried out by him, is *accepted as the P.G. Major Project-I Work report submission* in partial fulfillment of the requirement for the award of the degree of Master of Technology in Information Technology in the Department of Information Technology, NITK Surathkal, India.

Prof. Ananthanarayana V. S.
(Signature of the guide with Date and Seal)

Chairman - DPGC
(Signature with Date and Seal)

ACKNOWLEDGEMENT

I would sincerely like to thank my guide, Prof. Ananthanarayana V. S., Department of Information Technology, for his constant guidance, encouragement and valuable feedback throughout this phase of the Major Project. His suggestions helped me shape the problem clearly and keep the work grounded and honest. I am also grateful to the Department of Information Technology, NITK Surathkal, for providing the resources and environment needed to carry out this work, and to my family and friends for their support.

Kshitij Khede

ABSTRACT

Retrieval-augmented generation (RAG) has become a standard way of grounding large language models in external text, but most RAG systems treat a document as a set of independent chunks or re-cluster it by embedding similarity. In doing so they discard the section hierarchy that the author of the document deliberately created, so a passage that is later retrieved often reaches the language model without the heading and parent section that give it meaning.

This project develops SHIA-RAG, a structure-aware RAG system that recovers a document's own hierarchy and keeps it intact during retrieval. Unlike recent tree- and graph-based methods, which build their structure using large language models or clustering, SHIA-RAG induces the hierarchy directly from the document's layout and section numbering, so the tree is built deterministically and without any per-document model calls. A semantic concept forest is then grown over this tree, giving a two-tier representation in which every retrieved concept can be traced back to its section and its ancestors.

This mid-semester report presents the problem formulation, a survey of the relevant literature, the proposed architecture and methodology, and the part of the system implemented so far: the ingestion, structural parsing, hierarchy-induction and concept-forest layers, together with an initial structure-aware retrieval path. A preliminary experiment on a fourteen-paper corpus shows that, under an equal retrieval budget, SHIA-RAG locates the target content far more often than a flat and a clustering-tree baseline while preserving parent context almost completely. The full end-task evaluation against competing methods is planned for the end-semester phase.

Keywords— Retrieval-Augmented Generation, Document Hierarchy, Structure-Aware Retrieval, Knowledge Forest, Precedence Constraint

CONTENTS

LIST OF FIGURES **iv**

LIST OF TABLES **vi**

ABBREVIATIONS **ix**

1 Introduction **1**

2 Literature Review **3**

2.1 Outcome of Literature Review 4

2.2 Problem Statement 5

2.3 Objectives 5

2.4 Summary 5

3 Methodology **7**

3.1 Document Ingestion and Reading Order 7

3.2 Hierarchy Induction 8

3.3 Dual-Tier Representation 8

3.4 Structure-Aware Retrieval 9

3.5 Summary 9

4 Work Done **11**

4.1 Experimental Setup 11

4.2 Results and Discussion 11

4.3 Summary 13

5 Conclusions and Future Work **15**

5.1 Conclusions 15

5.2 Future Work 15

REFERENCES **17**

LIST OF FIGURES

1.1 SHIA-RAG end-to-end pipeline. The shaded stages are the core contributions of this work. 2

3.1 The nine-layer SHIA-RAG architecture. Shaded layers are implemented in the current phase. 7
3.2 A document tree recovered from section numbering; edges encode parent-to-child precedence. 8
3.3 Dual-tier representation: concepts are grounded in the document tree, so each retrieved concept can be traced back to its section and parent context. 9

4.1 Preliminary retrieval comparison on the fourteen-paper corpus under an equal budget (no LLM). 12

LIST OF TABLES

2.1 Summary of literature survey for the most relevant related works . . 4

4.1 Preliminary retrieval comparison on the fourteen-paper corpus (equal budget, no LLM) 12

ABBREVIATIONS

RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
QA	Question Answering
TF-IDF	Term Frequency–Inverse Document Frequency
PAA	Parent-Assignment Accuracy
DAG	Directed Acyclic Graph
SRDR	Semantic-Routing Depth Router
KCE	Knowledge Confidence Estimation

Chapter 1

Introduction

Large language models can produce fluent answers, but on their own they cannot be trusted to state facts about a specific document or a private collection of documents. Retrieval-augmented generation (RAG) [1] addresses this by first fetching relevant text from a store and then asking the model to answer using that text. The quality of the final answer therefore depends heavily on the retrieval step: if the wrong passages are fetched, or the right passage is fetched without the context that explains it, the answer suffers no matter how capable the model is.

Most documents that people actually work with – research papers, manuals, reports, contracts – are not flat. They are organised into a hierarchy of sections and subsections, and the meaning of a paragraph often depends on the heading above it. A sentence such as "we use a batch size of 32" is only meaningful once we know it sits under "Experimental Setup" rather than "Related Work". Common RAG pipelines, however, cut a document into fixed-size chunks and index them independently, so this structural context is lost at the very first step. More recent methods rebuild some structure – by clustering embeddings or by asking a large model to parse the document – but they do so at the cost of extra model calls, and the structure they recover is not always the document's real one.

This motivates a different approach. A document already carries its structure on its surface: numbered headings, indentation and reading order. If this structure can be recovered directly and cheaply, retrieval can be made to respect it, returning not just a relevant passage but the passage together with the section it belongs to. The system proposed in this project, SHIA-RAG, is built on exactly this idea. Figure 1.1 shows the overall flow: a document is ingested and ordered, its hierarchy is induced, a semantic concept forest is grown over it, and retrieval then draws on this structure to hand the reader evidence with its parent context intact.

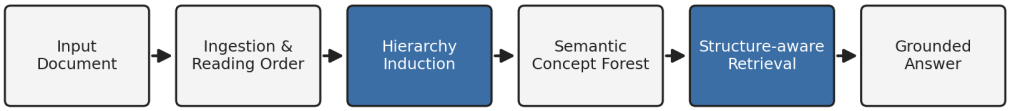


Figure 1.1: SHIA-RAG end-to-end pipeline. The shaded stages are the core contributions of this work.

The central idea of SHIA-RAG is to separate the two things that most systems entangle. The structure of a document – which section a passage belongs to and which sections contain it – is recovered once, directly from the layout and numbering, and stored as an explicit tree. The meaning of the text is captured separately as a forest of concepts grown over that tree. Because the structure is fixed and cheap to obtain, retrieval can be made to respect it without paying for a language model at indexing time, and every piece of evidence it returns can be traced back to the section it came from. This makes the retrieved context not only more relevant but also auditable, since a reader (or an examiner) can see exactly where each retrieved fact sits in the original document.

This report covers the first phase of the Major Project. The emphasis here is on framing the problem precisely, surveying how existing methods handle – or fail to handle – document structure, and building and validating the parts of the system that recover the hierarchy. A small, reproducible experiment is included to show that the structural machinery behaves as intended; the full comparison against competing systems on standard question-answering benchmarks is the subject of the end-semester phase. Keeping the pipeline runnable and measurable from an early stage is a deliberate choice, so that each later stage deepens a working system rather than assembling one at the end.

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature, states the outcome of that review, and fixes the problem statement and objectives of the project. Chapter 3 describes the proposed architecture and the methodology for hierarchy induction and structure-aware retrieval. Chapter 4 reports the implementation completed so far and a preliminary result. Chapter 5 draws the conclusions of this phase and outlines the future work.

Chapter 2

Literature Review

The work related to this project falls into three groups: the foundational RAG methods, the more recent structure-aware (tree and graph) methods, and the self-reflective or agentic methods.

The idea of coupling a parametric model with a non-parametric text store was popularised by Lewis et al. [1], whose RAG model retrieves passages with a dense retriever and conditions generation on them. Earlier, REALM [2] introduced retrieval into language-model pre-training, and RETRO [3] showed that retrieval could scale to very large token stores; Atlas [4] later demonstrated strong few-shot performance from a retrieval-augmented model. These methods established the retrieve-then-read pattern but treat the corpus as a flat set of passages, so the internal structure of any single document plays no role in how evidence is selected.

A second line of work tries to add structure back. RAPTOR [5] builds a tree by recursively clustering and summarising chunks and retrieves from a collapsed view of that tree. GraphRAG [7] constructs an entity–relationship graph over the corpus and answers using community summaries. TreeRAG [8] parses a document into a title tree and prepends ancestor titles during retrieval, while HiChunk [9] learns to split a document into a multi-level chunk tree and merges children into parents when needed. DTE-RAG [10] maintains two trees – one semantic, one lexical – for multi-hop reasoning, and T-Retriever [11] recasts graph retrieval as tree-based retrieval over textual graphs. These methods confirm that structure helps, which supports the direction of this project; however, they build their structure either by clustering embeddings, which does not follow the document's real sections, or by invoking a large or fine-tuned language model for every document, which is expensive and non-deterministic.

A third group focuses on control and self-correction rather than structure. Self-RAG [6] trains a model to decide when to retrieve and to critique its own output, and recent agentic approaches add planning, reranking and groundedness checks around the retriever. These ideas are relevant to the later stages of this project but are largely orthogonal to the structural problem tackled here. Table 2.1 summarises the most relevant works.

Table 2.1: Summary of literature survey for the most relevant related works

Work	**Methodology**	**Observation / limitation**
RAPTOR (ICLR 2024)	Recursive clustering + summarisation into a tree; collapsed-tree retrieval	Does not follow the real section structure; needs an LLM summary per node
TreeRAG (ACL 2025)	LLM parses a document into a title tree; ancestor titles prepended	Uses a large LLM to build the tree for every document
HiChunk (2025)	Fine-tuned model splits text into a multi-level chunk tree; auto-merge	Requires a fine-tuned model to recover structure
DTE-RAG (IEEE 2026)	Dual (semantic + lexical) trees for multi-hop reasoning	Trees are signal-based, not the document's own hierarchy
GraphRAG (2024)	Entity–relationship graph + community summaries	Graph, not document hierarchy; heavy indexing cost

2.1 Outcome of Literature Review

Across all three groups, no existing method recovers a document's true hierarchy cheaply and then guarantees that retrieved evidence carries its parent context. Clustering methods lose the real structure, LLM-based parsers recover it but at a cost, and self-reflective methods do not address structure at all. This gap – a training-free way to build a document's actual tree from its layout, together with a retrieval step constrained to preserve ancestor context – is what this project addresses.

2.2 Problem Statement

Existing retrieval-augmented generation systems discard a document's section hierarchy, so the retrieved evidence loses the parent context needed to interpret it correctly.

2.3 Objectives

1. To study existing RAG approaches – flat, tree-based, graph-based and dual-tree – and identify the loss of document structure and parent context as a shared limitation.

2. To design a training-free architecture (SHIA-RAG) that induces a document's hierarchy from its own layout and section numbering, without any per-document language-model calls.

3. To implement the document ingestion, structural parsing and hierarchy-induction layers that build and validate a document tree.

4. To construct a dual-tier representation by extracting a semantic concept forest, with typed relations and confidence, over the document tree.

5. To develop a structure-aware, precedence-constrained retrieval mechanism that keeps parent and ancestor context within a fixed token budget.

6. To evaluate SHIA-RAG against representative RAG baselines on measures of structural fidelity and retrieval quality, and to analyse the results.

2.4 Summary

This chapter reviewed the RAG literature, identified the loss of structure and parent context as the common gap, and fixed the problem statement and six objectives of the project. The next chapter describes how SHIA-RAG is designed to close this gap.

Chapter 3

Methodology

SHIA-RAG is organised as a stack of layers, shown in Figure 3.1. The lower layers turn a raw document into a validated tree and a concept forest; the middle layer retrieves from this representation under a budget; and the upper layers generate a grounded answer and, in later phases, let the system improve from feedback. The design principle throughout is that structure should be recovered once, cheaply and deterministically, and then respected at every subsequent step.

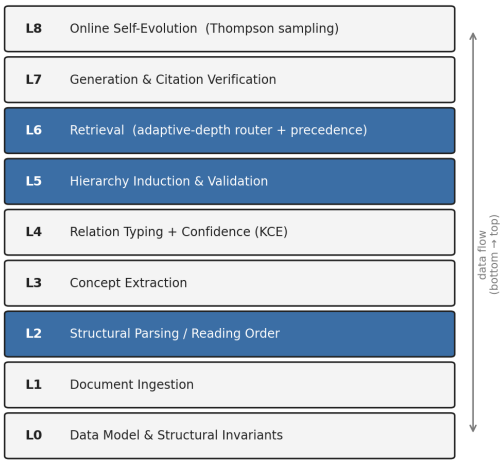


Figure 3.1: The nine-layer SHIA-RAG architecture. Shaded layers are implemented in the current phase.

3.1 Document Ingestion and Reading Order

The ingestion layer reads a document and produces an ordered list of text blocks. For multi-column material the naive top-to-bottom order is wrong, because it interleaves the two columns; the parser therefore uses the column gutter to sort blocks into the correct human reading order before any structure is inferred. Each block keeps its position and, where present, its section label, so that later layers can reason about order and nesting.

3.2 Hierarchy Induction

The core of the present phase is hierarchy induction – assigning every block its correct parent so that the document becomes a tree. The primary signal is the document's own section numbering: a block headed "3.1" is attached under "3". Where explicit numbering is absent, a fallback ranks candidate parents by a combination of structural position, textual similarity and heading level, and attaches the block to the best earlier, shallower heading. The result is validated against a set of invariants – the graph must remain acyclic, every non-root block must receive exactly one parent, and no block may be left orphaned. Figure 3.2 shows a small example of the tree produced. Because no language model is used to build the tree, the process is deterministic, auditable and fast, and it follows the structure the author actually wrote rather than an approximation inferred from embeddings.

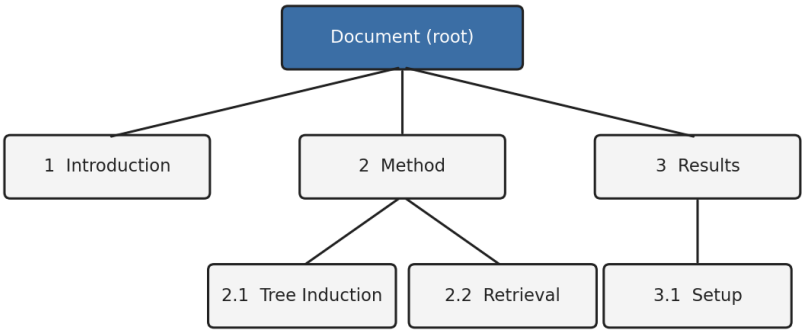


Figure 3.2: A document tree recovered from section numbering; edges encode parent-to-child precedence.

3.3 Dual-Tier Representation

On top of the document tree, the system extracts a semantic concept forest and links each concept back to the block and section it came from. This gives the dual-tier representation illustrated in Figure 3.3: a structural tier that preserves the document's sections and a semantic tier that captures the concepts, with every concept anchored in the structure. This anchoring is what later lets retrieval return a concept together with its parent context.

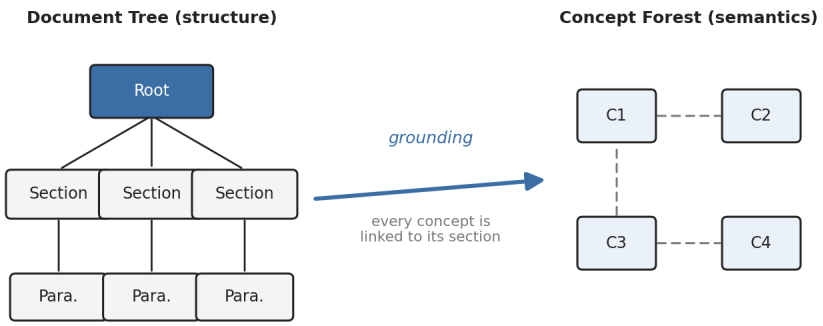


Figure 3.3: Dual-tier representation: concepts are grounded in the document tree, so each retrieved concept can be traced back to its section and parent context.

3.4 Structure-Aware Retrieval

Retrieval is designed to exploit the tree rather than ignore it. Given a question, the system first identifies the blocks most relevant to it, and then uses the tree to attach the parent and ancestor sections of those blocks, so that the evidence handed to the language model always arrives with the context that frames it. The amount of text is kept within a fixed token budget, and the selection is constrained so that a child is never returned without a path to its ancestors – the precedence constraint. The detailed retrieval mechanism and its evaluation form the main work of the end-semester phase.

3.5 Summary

This chapter presented the layered architecture and the methodology for the current phase: reading-order recovery, training-free hierarchy induction with validation, and the dual-tier representation, together with the design of structure-aware retrieval. The next chapter reports what has been implemented and a preliminary result.

Chapter 4

Work Done

4.1 Experimental Setup

The system is implemented in Python as a layered package. The data model and its invariants, document ingestion, structural parsing and reading-order recovery, concept extraction, relation typing with confidence, and hierarchy induction with validation are all in place and run end to end on real documents. An initial retrieval path is also working and is currently being extended into the full precedence-constrained mechanism. The implementation relies on lightweight, deterministic components – geometric parsing and standard vector and numerical libraries – rather than heavy external services, which keeps the pipeline reproducible on a single machine. For the preliminary experiment reported below, a corpus of fourteen research papers was used, with a set of realistic queries and an equal, tight retrieval budget applied to every method so that the comparison reflects retrieval behaviour alone.

4.2 Results and Discussion

To check whether the structural design behaves as intended, an initial experiment compared three retrieval strategies under the same budget: SHIA-RAG, a RAPTOR-style clustering-tree retriever, and a flat top-k retriever. Because this experiment measures retrieval behaviour rather than final answer quality, it needs no language model, which makes it fully reproducible. Two quantities were measured: how often the retriever located the intended target content, and how often it kept the parent or ancestor context of what it returned. The results are shown in Figure 4.1 and Table 4.1.

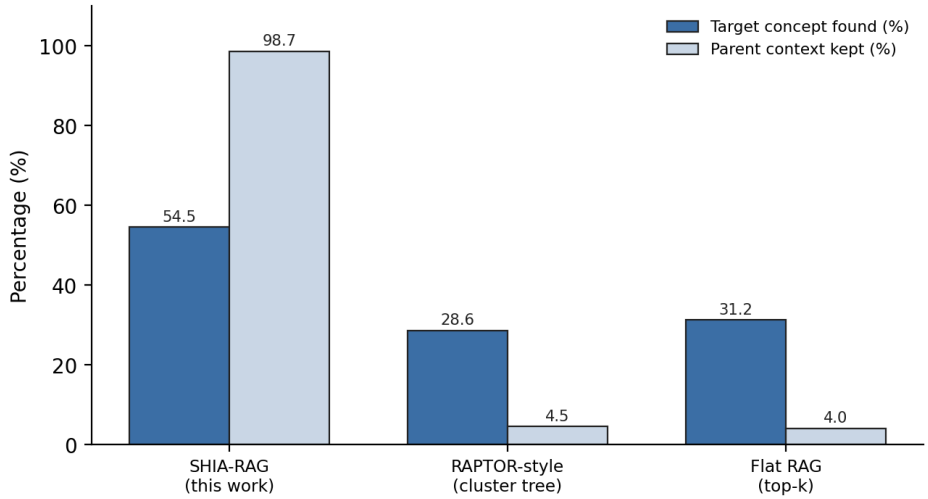


Figure 4.1: Preliminary retrieval comparison on the fourteen-paper corpus under an equal budget (no LLM).

Table 4.1: Preliminary retrieval comparison on the fourteen-paper corpus (equal budget, no LLM)

Method	**Target found (%)**	**Parent context kept (%)**
SHIA-RAG (this work)	**54.5**	**98.7**
RAPTOR-style (clustering tree)	28.6	4.5
Flat RAG (top-k)	31.2	4.0

The pattern is clear and matches the design intent. SHIA-RAG found the target content markedly more often than either baseline, and it preserved parent context almost completely (98.7%), against roughly four to five per cent for the clustering-tree and flat methods. The reason is structural: the clustering-tree groups text by similarity and so does not follow the document's real sections, while flat retrieval has no notion of a parent at all; SHIA-RAG follows the induced tree and enforces precedence, so the ancestor context is retained by construction. This is a preliminary, retrieval-level result and is deliberately limited in scope. It shows that the structural machinery works, which is the milestone for this phase; it does not yet measure end-task question-answering accuracy against the full set of competing systems, which is the main deliverable of the end-semester phase.

4.3 Summary

The foundational layers of SHIA-RAG have been implemented and shown to run on real documents, and a preliminary experiment confirms that the structural retrieval behaves as intended, preserving parent context far better than flat and clustering-tree baselines. The conclusions drawn from this phase and the plan for the remaining work are given in the next chapter.

Chapter 5

Conclusions and Future Work

5.1 Conclusions

This report addressed the loss of document structure in retrieval-augmented generation and proposed SHIA-RAG as a training-free, structure-aware solution. Over the course of this phase, the problem was framed precisely, the relevant literature was surveyed, and the common gap – the failure of existing methods to recover a document's real hierarchy cheaply and to preserve parent context during retrieval – was identified. A layered architecture was then designed around a single principle: recover a document's hierarchy once, directly from its layout and section numbering, and respect that hierarchy at every later step.

The foundational layers of this architecture have been implemented and shown to run end to end on real documents: document ingestion, reading-order recovery, training-free hierarchy induction with validation, and the dual-tier concept forest that links every concept back to its section. A preliminary, reproducible experiment on a fourteen-paper corpus, run under an equal retrieval budget and without any language model, showed that SHIA-RAG locates the intended target content far more often than a clustering-tree and a flat baseline (54.5% against 28.6% and 31.2%) and preserves parent context almost completely (98.7% against roughly four to five per cent). This confirms that the structural machinery works, and that the central claim of the project – that a document's own hierarchy can be recovered cheaply and used to keep retrieved evidence in context – holds in practice for the part of the system built so far.

5.2 Future Work

The work planned for the end-semester phase is a direct continuation of what has been built, and is limited to what the current system makes achievable:

1. Complete the precedence-constrained retrieval mechanism, so that query-relevant leaf passages and their ancestor context are combined within a fixed token budget.

2. Carry out a fair, like-for-like evaluation of SHIA-RAG against representative RAG baselines – RAPTOR, TreeRAG, GraphRAG, HiChunk and DTE-RAG – on standard question-answering benchmarks, using one shared reader model and an equal retrieval budget for every method, so that any difference reflects retrieval quality alone.

3. Extend the structural-fidelity measurement begun in this phase to a larger set of documents, to confirm that the parent-context advantage is stable.

4. Conduct an ablation that removes the precedence constraint, to verify that it is what accounts for the structural advantage.

These tasks reuse the pipeline and the evaluation harness already prepared in this phase, and together they complete the objectives set out in Chapter 2.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," *International Conference on Machine Learning (ICML)*, 2020.

[3] S. Borgeaud, A. Mensch, J. Hoffmann, et al., "Improving Language Models by Retrieving from Trillions of Tokens," *ICML*, 2022.

[4] G. Izacard, P. Lewis, M. Lomeli, et al., "Atlas: Few-shot Learning with Retrieval Augmented Language Models," *Journal of Machine Learning Research*, 2023.

[5] P. Sarthi, S. Abdullah, A. Tuli, et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *International Conference on Learning Representations (ICLR)*, 2024.

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," *ICLR*, 2024.

[7] D. Edge, H. Trinh, N. Cheng, et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv preprint, 2024.

[8] "TreeRAG: Hierarchical Title-Tree Retrieval for Structured Documents," *Association for Computational Linguistics (ACL)*, 2025.

[9] "HiChunk: Hierarchical Document Chunking for Retrieval-Augmented Generation," 2025.

[10] "DTE-RAG: A Dual-Tree Structure for Multi-Hop Reasoning," *IEEE ISBDAS*, 2026.

[11] "T-Retriever: Tree-based Hierarchical Retrieval for Textual Graphs," arXiv preprint, 2026.